

HATCom:An Advanced Community Detection Framework for Complex Network Analysis

*Project report submitted to the Amrita Vishwa Vidyapeetham in partial
fulfilment of the requirement for the Degree of*

BACHELOR of TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Anabala Balaji	AM.EN.U4CSE22305
Koneti Manoj	AM.EN.U4CSE22329
M Karthikeya Sarvan	AM.EN.U4CSE22334
Siva Ganga K M	AM.EN.U4CSE22352



**AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
(Estd. U/S 3 of the UGC Act 1956)
AMRITAPURI CAMPUS
KOLLAM - 690525**

MAY 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMRITA VISHWA VIDYAPEETHAM
(Estd. U/S 3 of the UGC Act 1956)
Amritapuri Campus
Kollam - 690525



BONAFIDE CERTIFICATE

This is to certify that the project report entitled “**HATCom: An Advanced Community Detection Framework for Complex Network Analysis**” submitted by **Anabala Balaji (AM.EN.U4CSE22305)**, **Koneti Manoj (AM.EN.U4CSE22329)**, **M Karthikeya Sarvan (AM.EN.U4CSE22334)** and **Siva Ganga K M (AM.EN.U4CSE22352)**, in partial fulfillment of the requirements for the award of Degree of Bachelor of Technology in Computer Science and Engineering from Amrita Vishwa Vidyapeetham, is a bonafide record of the work carried out by them under the guidance of **Deepthi L. R.** and coordination of **Thara S**, at Amrita School of Computing, Amritapuri during Semester 8 of the academic year 2025–2026.

Project Guide
Deepthi L. R.

Project Coordinator
Thara S

Chairperson, Dept. of CSE

Reviewer

Place : Amritapuri
Date : 18 May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMRITA VISHWA VIDYAPEETHAM
(Estd. U/S 3 of the UGC Act 1956)
Amritapuri Campus
Kollam - 690525



DECLARATION

We, **Anabala Balaji (AM.EN.U4CSE22305), Koneti Manoj (AM.EN.U4CSE22329), M Karthikeya Sarvan (AM.EN.U4CSE22334) and Siva Ganga K M (AM.EN.U4CSE22352)**, hereby declare that this project entitled **“HATCom: An Advanced Community Detection Framework for Complex Network Analysis”** is a record of the original work done by us under the guidance of **Deepthi L. R.**, Dept. of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, and that this work has not formed the basis for the award of any degree/diploma/fellowship or similar awards to any candidate in any university to the best of our knowledge.

Place : Amritapuri
Date : 18 May 2026

Signature of the Students

Signature of the Project Guide

Acknowledgements

We would like to express our sincere gratitude to our Project Guide, **Deepthi L. R.**, for her continuous guidance, support, and encouragement throughout this project. Her valuable suggestions and insights played an important role in shaping our work and helped us successfully complete the project. We are also thankful to our Project Coordinator, **Thara S**, and the faculty members of the Department of Computer Science and Engineering, Amrita School of Computing, for their support and guidance. We extend our heartfelt thanks to Amrita Vishwa Vidyapeetham, Amritapuri Campus, for providing the necessary infrastructure and a supportive environment for carrying out this project. Finally, we would like to thank our family for their constant encouragement, trust, and support, without which this work would not have been possible.

The Authors

Abstract

Detecting communities is a notoriously difficult task in the real-world data due to their node types heterogeneity, rich attributed network and the presence of temporal dynamics. Conventional approaches for community detection generally focus on static and homogeneous graphs and hence fail to capture the complex nature of real-world datasets. Our project **HATCom** presents a unified framework for community detection in Heterogeneous, Attributed, and Temporal Networks. HATCom models the real-world data as temporal heterogeneous graph which consists of various node types such as user, movie, and genre, and includes different relationships along with time-varying interactions. We introduce a **Relation-Time Co-Attention (RTCA)** mechanism to consider the relation based semantic knowledge and temporal recentness. We use **Graph Transformer Network (GTN)** to learn useful meta-path representations. Additionally, tracking evolving communities requires temporal stability. To handle this, our project introduces a memory module driven by an Exponential Moving Average (EMA), which stabilizes node representations over time. We evaluated HATCom using Normalised Mutual Information (NMI), Adjusted Rand Index (ARI), and Modularity scores. The results confirm that our framework effectively isolates meaningful community structures despite the inherent complexity of temporal networks.

Contents

Contents	ii
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Study/Project	3
2 Problem Definition	4
2.1 Overview	4
2.2 Formal Problem Formulation	4
2.2.1 Heterogeneous Attributed Temporal Graph	4
2.2.2 Node Embedding	5
2.2.3 Community Detection	6
2.3 Key Concepts and Definitions	6
2.4 Research Challenges	6
2.4.1 Challenge 1: Simultaneous Modelling of Heterogeneity, Attributes, and Temporal Dynamics	6
2.4.2 Challenge 2: Automatic Meta-Path Discovery	7

2.4.3	Challenge 3: Temporal Stability of Community Assignments	8
2.4.4	Challenge 4: Scalability to Large Heterogeneous Graphs	8
2.4.5	Challenge 5: Evaluation Without Ground-Truth Labels	8
2.5	Problem Scope and Constraints	9
2.6	Distinction from Related Problems	10
3	Literature Review	11
3.1	Purpose and Importance	11
3.2	Various Studies	11
3.3	Approach	13
4	Requirements	15
4.1	Hardware Requirements	15
4.1.1	Minimum Requirements	15
4.2	Software Requirements	16
4.2.1	Operating System	16
4.2.2	Programming Environment	16
4.2.3	Python Libraries	16
4.3	Dataset Description	16
5	Proposed System	18
5.1	System Overview	18
5.2	Graph Construction and Data Preparation	19
5.2.1	Dataset and Preprocessing	19
5.2.2	Graph Schema	20
5.3	Type-Aware Attribute Encoding	21
5.3.1	User Node Encoding	22
5.3.2	Movie Node Encoding	22
5.3.3	Genre Node Encoding	23
5.4	Relation-Time Co-Attention (RTCA)	23
5.4.1	Motivation	23

5.4.2	Temporal Encoding	24
5.4.3	Attention Score Computation	24
5.4.4	Neighbourhood Aggregation	25
5.4.5	RTCA Algorithm	25
5.5	Graph Transformer Network (GTN)	25
5.5.1	Motivation	25
5.5.2	Multi-Channel Aggregation	26
5.5.3	GTN Algorithm	27
5.6	EMA-Based Memory Module	27
5.6.1	Motivation	27
5.6.2	Temporal Snapshot Processing	28
5.6.3	Memory Module Algorithm	28
5.7	Self-Supervised Training	29
5.7.1	Link Prediction Objective	29
5.7.2	Training Configuration	30
5.7.3	Training Algorithm	31
5.8	Community Detection	31
5.9	System Architecture Summary	31
6	Result and Analysis	33
6.1	Experimental Setup	33
6.1.1	Dataset	33
6.1.2	Evaluation Metrics	34
6.1.3	Baselines	34
6.2	Community Detection Performance	35
6.3	Temporal Stability Analysis	36
6.4	Genre Classification Performance	38
6.5	Link Prediction Performance	39
6.6	Baseline Comparison	40
6.7	Visualisation and Analysis	41
6.7.1	t-SNE Embedding Visualisation	41
6.7.2	RTCA Attention Weight Analysis	42

6.8	Ablation Study	43
6.9	Key Insights and Discussion	45
7	Conclusion	46
7.1	Summary of Contributions	46
7.2	Summary of Results	47
7.3	Limitations	48
7.4	Future Work	48
7.5	Concluding Remarks	49
	References	50
A	Source Code	53
A.1	Relation-Time Co-Attention (RTCA) Module	53
A.2	Graph Transformer Network (GTN) Module	54
A.3	EMA Memory Module	55
A.4	Self-Supervised Training Loop	56

List of Figures

5.1	HATCom system pipeline overview.	19
5.2	Heterogeneous graph structure of the MovieLens 100K dataset.	21
5.3	Temporal memory summary across five snapshots.	29
6.1	Snapshot-to-snapshot community stability (NMI).	37
6.2	Evolution of top-6 cluster sizes across 5 temporal snapshots. .	37
6.3	Cumulative mean NMI stability across snapshot transitions. .	38
6.4	t-SNE visualisation of learned HATCom movie embeddings. .	41
6.5	RTCA attention weights across edge types and temporal bins.	42
6.6	Recency-weighted semantic importance per edge type and tem- poral bin.	43

List of Tables

2.1	Key Concepts and Definitions	7
3.1	Comparison of Existing Methods and HATCom	14
5.1	HATCom Architecture Summary	32
6.1	MovieLens 100K Dataset Statistics	33
6.2	Community Detection Performance of HATCom	35
6.3	Genre Classification Using Learned Movie Embeddings (KNN, $k = 5$)	39
6.4	Link Prediction Performance on Validation Set	39
6.5	Comparison Table	40
6.6	Ablation Study Results Across Clustering, Link Prediction, and Genre Classification Metrics	44

Chapter 1

Introduction

1.1 Background

Community detection is a crucial task in network analysis, it is generally used to identify groups of nodes that share similar structural or semantic properties. It plays a crucial role in applications such as social network analysis and knowledge discovery. Traditional community detection models usually treat all nodes and relationships uniformly, focusing heavily on static, homogeneous graphs.

In contrast, real-world networks are far more complex because they constantly evolve and contain diverse entities with rich attributes. For example, in movie platforms, users interact with movies based on preferences that might change over time and these interactions can be observed by contextual attributes such as genre preferences and ratings.

To effectively model such systems, it is necessary to consider three key aspects simultaneously: heterogeneity, attributes, and temporal dynamics. These three aspects dictate complete representations and meaningful community structures. This indicates the need for frameworks capable of capturing the full complexity of real-world networks.

1.2 Problem Statement

Modern community detection systems face several challenges when applied to real-world data. Existing methods often fail to capture the heterogeneous nature of networks, where multiple node and edge types coexist. Additionally, many approaches ignore temporal evolution, treating all interactions as static and equally important, which leads to outdated or inaccurate community assignments.

Furthermore, traditional models struggle to incorporate attribute information effectively, resulting in a loss of semantic understanding. Many methods also rely on rigid clustering techniques that do not support overlapping communities, even though real-world entities often belong to multiple groups simultaneously.

Therefore, there is a need for a unified framework that can effectively model heterogeneous, attributed, and temporal networks while providing meaningful, interpretable, and scalable community detection.

1.3 Objectives

The main objective of this project is to design and develop a unified framework for community detection in heterogeneous, attributed, and temporal networks.

Specific Objectives:

- To construct a temporal heterogeneous graph from real-world complex datasets (e.g. MovieLens).
- To incorporate attribute information for multiple node types including users, movies, and genres.
- To develop a relation-aware and time-aware representation learning mechanism.

- To implement advanced modules such as Relation-Time Co-Attention (RTCA) and Graph Transformer Networks (GTN).
- To enable detection of soft and overlapping communities.
- To evaluate the performance using clustering metrics such as NMI, ARI, and Modularity.

1.4 Scope of the Study/Project

The scope of this project focuses on modeling real-world interaction data using a temporal heterogeneous graph framework. It takes into consideration multiple types of nodes, attributes, and time-based interactions to create meaningful node embeddings and enable the discovery of communities.

The model is built using Python frameworks such as PyTorch, DGL, and NetworkX, and tests are performed on the MovieLens dataset. In line with its goals of interpretation, scalability, and unsupervised learning, the project finds application in recommendation systems and network analysis.

However, the research is limited to offline processing only. Deploying it in production settings, where there are large amounts of streaming data to process, is out of scope.

Chapter 2

Problem Definition

2.1 Overview

Community detection studies have been significantly developed over the last couple of decades. The earliest approaches dealt with community detection as a graph partitioning problem for simple, static, and homogeneous graphs. Recent advances in digital communication highlight the true complexity of real-world networks, which feature diverse node and edge types, rich features, and continuous temporal evolution. Ignoring any of these dimensions during community detection results in representations that are incomplete, outdated, or semantically shallow.

This chapter defines the problem addressed by the HATCom. It introduces the key concepts and mathematical notation used throughout the report, and explains research challenges that motivated the proposed design.

2.2 Formal Problem Formulation

2.2.1 Heterogeneous Attributed Temporal Graph

We define a Heterogeneous Attributed Temporal Graph (HATG) as $G = (V, E, X, T)$. In this structure, $V = \{v_1, v_2, \dots, v_n\}$ represents the set of

nodes where each node v_i is assigned a specific type $\tau(v_i) \in \mathcal{T}_V$, such as users, movies, or genres. In the context of the MovieLens dataset, $\mathcal{T}_V = \{\text{User, Movie, Genre}\}$.

These nodes are connected by the edge set $E = \{e_1, e_2, \dots, e_m\}$, with each edge $e_k = (v_i, v_j, r, t)$ capturing a specific relationship type and timestamp. In the MovieLens graph, $\mathcal{T}_E = \{\text{rates, has_genre, user_similarity, shares_genre}\}$. Furthermore, the graph is attributed because each node carries a feature vector $X = \{x_{v_i} \mid v_i \in V\}$, and it is temporal because the timestamp set $T = \{t_k \mid e_k \in E\}$ records the exact chronological order of every interaction. The graph is *heterogeneous* because $|\mathcal{T}_V| > 1$ and $|\mathcal{T}_E| > 1$. It is *attributed* because of the fact that each node has a feature vector x_{v_i} associated with it. It is *temporal* because each edge carries a timestamp t_k that encodes when the interaction occurred.

2.2.2 Node Embedding

Given the HATG G , the task of representation learning consists in deriving a lower-dimensional vector representation $\mathbf{h}_{v_i} \in \mathbb{R}^d$ of each node $v_i \in V$, such that the embedding captures:

1. The structural neighbourhood of v_i within the graph.
2. The attribute information x_{v_i} associated with v_i .
3. The relational semantics of the edges connecting v_i to its neighbours.
4. The temporal recency of interactions, giving higher weight to more recent edges.

Mathematically, the embedding function can be defined as:

$$\mathbf{h}_{v_i} = f_{\theta}(v_i, \mathcal{N}(v_i), X, T, \mathcal{T}_E) \quad (2.1)$$

where $\mathcal{N}(v_i)$ represents the v_i , and f_{θ} is a parameterised function learned during training.

2.2.3 Community Detection

Given the learned embeddings $\mathbf{H} = \{\mathbf{h}_{v_i} \mid v_i \in V\}$, the community detection problem is to assign each node to one or more communities. Formally, a community assignment is a function:

$$C : V \rightarrow 2^{\{1,2,\dots,K\}} \quad (2.2)$$

where K is the number of communities and $2^{\{1,\dots,K\}}$ denotes the power set, allowing each node to belong to multiple communities simultaneously (overlapping community detection).

The quality of the community assignment is measured using clustering metrics including Silhouette Score, Calinski-Harabasz Index, Davies-Bouldin Index, and Modularity, as defined in Chapter 5.

2.3 Key Concepts and Definitions

Table 2.1 summarises the core concepts used throughout this report.

2.4 Research Challenges

The problem of community detection in heterogeneous, attributed, and temporal networks presents several specific research challenges that are addressed by the HATCom framework:

2.4.1 Challenge 1: Simultaneous Modelling of Heterogeneity, Attributes, and Temporal Dynamics

Existing methods typically address one or two of these dimensions in isolation. Methods designed for heterogeneous graphs often ignore temporal dynamics. Methods designed for temporal graphs often assume homogeneous node types. Methods that incorporate attributes often do so in a

Table 2.1: Key Concepts and Definitions

Concept	Definition
Heterogeneous Graph	A graph with multiple node types ($ \mathcal{T}_V > 1$) and/or edge types ($ \mathcal{T}_E > 1$), also called a Heterogeneous Information Network (HIN).
Meta-Path	A sequence of node and edge types $\tau_1 \xrightarrow{\phi_1} \tau_2 \cdots \xrightarrow{\phi_l} \tau_{l+1}$ defining a composite relation. HATCom learns these automatically via GTN.
Temporal Graph	A graph where each edge carries a timestamp. HATCom uses continuous timestamps in RTCA and snapshot-based processing in the memory module.
Overlapping Community Detection	Each node may belong to multiple communities simultaneously. HATCom supports this via soft K-means on learned embeddings.
Self-Supervised Learning	Training on a pretext task (link prediction) without manual labels, forcing the model to learn structurally and semantically meaningful representations.

post-hoc manner that does not integrate seamlessly with the graph learning process. HATCom addresses this challenge by designing a unified architecture in which heterogeneity, attributes, and temporal dynamics are all first-class citizens of the model.

2.4.2 Challenge 2: Automatic Meta-Path Discovery

Manual meta-path definition requires deep domain expertise and is not scalable to new domains or datasets. Moreover, the optimal set of meta-paths for a given task is not always known in advance. HATCom addresses this challenge by using a Graph Transformer Network (GTN) that learns soft meta-paths automatically from data, eliminating the need for manual specification.

2.4.3 Challenge 3: Temporal Stability of Community Assignments

In dynamic networks, node embeddings can change rapidly as new interactions arrive, leading to unstable community assignments that fluctuate from one time step to the next. This instability makes it difficult to track the evolution of communities over time and can lead to poor user experience in applications such as recommendation systems. HATCom addresses this challenge by using an Exponential Moving Average (EMA) based memory module that smooths node embeddings across time, ensuring that community assignments remain stable in the absence of significant structural changes.

2.4.4 Challenge 4: Scalability to Large Heterogeneous Graphs

Heterogeneous graphs with multiple node types and edge types can be extremely large. The MovieLens 100K graph, for example, contains 2,644 nodes and approximately 1.85 million edges. Naive implementations of graph neural networks that require full graph materialisation or dense matrix operations cannot scale to such graphs. HATCom addresses this challenge through edge subsampling during training, $O(1)$ temporal encoding using exponential decay, and efficient attention computation that avoids quadratic complexity.

2.4.5 Challenge 5: Evaluation Without Ground-Truth Labels

Community detection is inherently an unsupervised task, and ground-truth community labels are rarely available in practice. Evaluating the quality of detected communities without labels requires the use of intrinsic clustering metrics such as Silhouette Score, Calinski-Harabasz Index, Davies-Bouldin Index, and Modularity. HATCom uses a comprehensive suite of such metrics, supplemented by genre classification accuracy as a proxy for semantic

coherence.

2.5 Problem Scope and Constraints

The problem addressed in this project is scoped as follows:

- **Dataset:** The MovieLens 100K dataset is used as the primary evaluation benchmark. It provides a rich combination of heterogeneous node types, temporal interaction data, and attribute information, making it well-suited for evaluating the proposed framework.
- **Graph Scale:** The graph contains 2,644 nodes (943 users, 1,682 movies, 18 genres) and approximately 1.85 million edges across four edge types. Training uses a subsampled set of 300,000 edges to manage computational constraints.
- **Community Granularity:** The number of communities K is set to 18, corresponding to the number of genre categories in the MovieLens dataset. This provides a natural ground truth for evaluating the semantic coherence of detected communities.
- **Temporal Resolution:** The dataset spans interactions from 1997 to 1998. The timeline is divided into 5 uniform temporal snapshots for the memory module and stability analysis.
- **Training Paradigm:** The model is trained in a fully unsupervised manner using self-supervised link prediction. No ground-truth community labels are used during training.
- **Evaluation:** The model is evaluated using five clustering metrics (Silhouette Score, Calinski-Harabasz Index, Davies-Bouldin Index, Modularity, NMI), link prediction AUC-ROC, and genre classification accuracy using a K-Nearest Neighbours classifier.

2.6 Distinction from Related Problems

HATCom is a community detection framework, not a node classifier, link predictor, graph generator, or recommendation system. While it uses link prediction as a self-supervised training signal and is evaluated on a recommendation dataset (MovieLens), the goal is unsupervised discovery of node groups — not predicting future interactions or assigning predefined labels to individual nodes.

Chapter 3

Literature Review

3.1 Purpose and Importance

The literature review plays a crucial role in understanding the existing advancements in community detection and identifying the research gaps in current systems. With the rapid growth of real-world data, networks have become increasingly complex, involving heterogeneous node types, rich attribute information, and temporal interactions. Traditional methods are not capable of handling all these aspects simultaneously.

A comprehensive study of existing techniques helps in evaluating their strengths and limitations, thereby enabling the design of improved models. It also assists in selecting appropriate methodologies, evaluation metrics, and system architectures for solving the problem effectively.

3.2 Various Studies

1. Node2Vec + Louvain (Static Methods):

Node2Vec [2] is a widely used graph embedding technique that generates node representations using biased random walks parameterised by return probability p and in-out probability q , allowing the model to interpolate between local (BFS) and global (DFS) neighbourhood exploration. These embed-

dings are commonly combined with the Louvain algorithm [3] for community detection based on modularity optimisation. Graph partitioning approaches such as the enhanced Girvan-Newman method [11] follow a similar divisive paradigm. While these approaches are computationally efficient and serve as strong baselines, they consider only structural information and ignore node attributes and temporal dynamics, making them unsuitable for real-world heterogeneous networks.

2. CS-TGN (Temporal Graph Networks):

Community Stability Temporal Graph Networks (CS-TGN) [5] introduce temporal modelling by capturing the evolution of node interactions over time through stability constraints that penalise abrupt changes in community assignments between consecutive time steps. These models ensure smooth community evolution and effectively handle dynamic changes in interaction patterns. Related work on spatio-temporal prediction using Graph Convolutional Networks [13] further demonstrates the effectiveness of GCN-based approaches in capturing temporal dependencies. However, CS-TGN and similar models treat all node types uniformly and fail to capture heterogeneous relationships, leading to reduced semantic understanding in multi-relational networks.

3. DyHANE (Heterogeneous Network Models):

Dynamic Heterogeneous Attributed Network Embedding (DyHANE) [4] focuses on integrating structural and attribute information in heterogeneous graphs with temporal dynamics. It uses a snapshot-based temporal model combined with a heterogeneous attention mechanism that considers multiple node types and relationships. Work on attribute-driven overlapping community detection [8] and quasi-attention on attributed heterogeneous graphs [7] further highlights the importance of node attributes in discovering meaningful communities. However, DyHANE relies on snapshot-based temporal modelling and faces scalability issues when applied to large and dense networks.

4. Graph Transformer Networks (GTN):

Graph Transformer Networks automatically learn meta-paths in heterogeneous graphs by combining edge-type adjacency matrices through a soft selection mechanism, enabling improved feature extraction and representation learning without manual meta-path specification [7]. CS-MLGCN [6] extends this direction with hierarchical graph convolution to capture multi-level community structures. Comparative studies on graph clustering algorithms [10] confirm that embedding quality is the primary bottleneck in clustering performance, motivating the focus on richer representation learning. However, these models do not fully incorporate temporal dynamics, limiting their effectiveness in evolving networks.

5. Attention-Based Graph Models:

Attention-based Graph Neural Networks [9] assign different importance weights to neighbouring nodes during message passing, improving the quality of learned representations and enabling the model to focus on the most relevant structural connections. Link prediction approaches using stochastic block modelling [12] and motif-based structural analysis [14] further demonstrate that higher-order relational patterns carry important information for community assignment. However, standard attention-based models often fail to integrate heterogeneity, node attributes, and temporal evolution simultaneously, resulting in incomplete modelling of real-world systems.

3.3 Approach

From the analysis of existing methods, it is evident that no single approach effectively integrates all key aspects of real-world networks like heterogeneity, temporal evolution, and attribute information. Most existing models address only one or two of these components, leading to incomplete representations and suboptimal community detection.

The proposed system, HATCom, addresses these limitations by introducing a unified framework that combines relation-aware attention, temporal mod-

elling, and automatic meta-path learning. The Relation-Time Co-Attention (RTCA) module captures both relational semantics and temporal recency, while the Graph Transformer Network (GTN) automatically learns meaningful meta-paths without manual intervention.

Additionally, a memory module based on exponential moving average ensures temporal stability in node representations. The system is trained using a self-supervised learning approach through link prediction [12], eliminating the need for labelled data.

This integrated approach enables the detection of meaningful, dynamic, and overlapping communities while maintaining scalability and interpretability, making it suitable for real-world applications such as recommendation systems and network analysis. Table 3.1 summarises how existing methods compare against HATCom.

Table 3.1: Comparison of Existing Methods and HATCom

Method	Hetero.	Attrs.	Temporal	Auto MP	Unsup.
Node2Vec + Louvain	✗	✗	✗	–	✓
CS-TGN	✗	✗	✓	–	✓
DyHANE	✓	✓	✓	✗	✗
CS-MLGCN	✓	✗	✗	✗	✗
GTN (original)	✓	✓	✗	✓	✗
HATCom	✓	✓	✓	✓	✓

Hetero. = Heterogeneous nodes/edges; *Attrs.* = Node attributes; *Auto MP* = Automatic meta-path discovery; *Unsup.* = Unsupervised training; – = not applicable.

Chapter 4

Requirements

4.1 Hardware Requirements

The proposed system involves both software and hardware requirements to implement the graph construction, model training, and evaluation pipeline. The hardware requirements are chosen to reflect a typical student or research lab environment that can handle graph-based computations.

4.1.1 Minimum Requirements

To handle the graph construction and training pipeline, the system requires a baseline setup typical of a research environment. At a minimum, this means an Intel Core i5 processor, 8 GB of RAM, and 20 GB of free storage. However, for efficient PyTorch training and faster data access, we strongly recommend stepping up to an Intel Core i7 with 16 GB of RAM, an SSD, and an NVIDIA GPU with CUDA support.

4.2 Software Requirements

4.2.1 Operating System

Commonly used modern operating systems such as Windows 11, macOS and many Linux Distributions can handle the proposed system.

4.2.2 Programming Environment

Python version 3.8 and above with IDEs such Jupyter Notebook and Google Colab are recommended.

4.2.3 Python Libraries

The following libraries are used for implementation:

- NumPy – for numerical computations
- Pandas – for data preprocessing and manipulation
- NetworkX – for heterogeneous graph construction
- PyTorch – for deep learning framework for RTCA and GTN modules
- Scikit-learn – for clustering and evaluation metrics (NMI, ARI)
- SentenceTransformers / Transformers – for textual feature encoding
- Matplotlib / Seaborn – for visualization of embeddings and results

4.3 Dataset Description

The system uses the MovieLens 100K dataset, a widely used benchmark dataset for recommendation systems and community detection tasks. It contains user interactions with movies along with ratings and timestamps.

The dataset includes the following components:

- Users – individuals interacting with the system
- Movies – items rated by users
- Genres – categories associated with movies

The dataset is preprocessed to:

- Map users, movies, and genres to node identifiers
- Construct user–movie rating edges with timestamps
- Build additional edges such as user–user similarity and movie–movie similarity

The processed data is represented as a heterogeneous attributed temporal graph (HATG), which serves as the input for the proposed HATCom framework.

Chapter 5

Proposed System

5.1 System Overview

HATCom (Heterogeneous, Attributed, and Temporal Community Detection) is a unified end-to-end framework designed to detect communities in complex real-world networks that simultaneously exhibit heterogeneity, rich attribute information, and temporal dynamics. HATCom relies on four core components working together: a Type-Aware Attribute Encoder, a Relation-Time Co-Attention (RTCA) module, a Graph Transformer Network (GTN), and an EMA-based Memory Module. Instead of relying on ground-truth community labels, the system trains these modules jointly through a self-supervised link prediction objective.

The overall pipeline of HATCom is illustrated in Figure 5.1. Raw data from the MovieLens 100K dataset is first preprocessed and mapped to a Heterogeneous Attributed Temporal Graph (HATG). Node features are then encoded using type-aware encoders that handle the diversity of attribute types across node categories. The encoded features are passed through the RTCA module, which aggregates neighbourhood information by simultaneously considering relation semantics and temporal recency. The output of RTCA is then processed by the GTN, which learns soft meta-paths automatically. The resulting embeddings are stabilised by the EMA memory module, and the final

embeddings are used for community detection and evaluation.

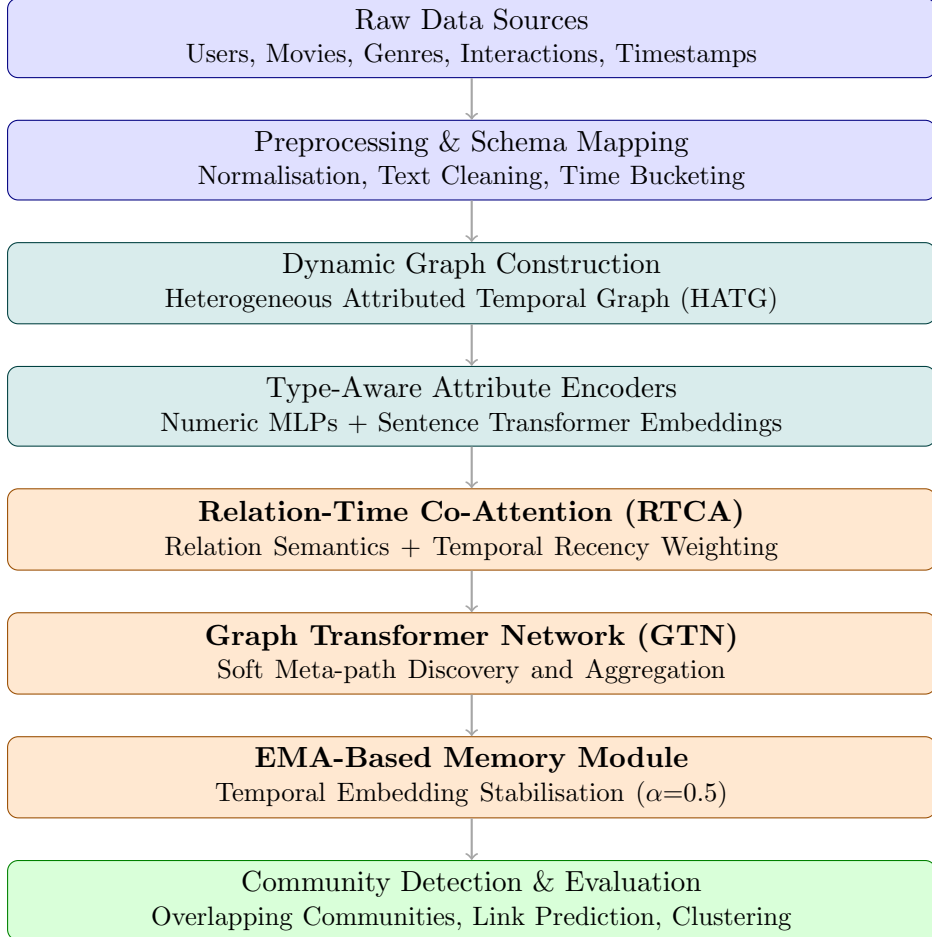


Figure 5.1: HATCom system pipeline. **Blue**: data ingestion and preprocessing. **Teal**: graph construction and feature encoding. **Orange**: core learned modules (RTCA, GTN, Memory). **Green**: community detection and evaluation output.

5.2 Graph Construction and Data Preparation

5.2.1 Dataset and Preprocessing

The system is built and evaluated on the MovieLens 100K dataset [1], a widely used benchmark in recommendation systems research. The dataset

contains 943 users, 1,682 movies, 100,000 ratings with timestamps, and 18 genre categories. Each user is described by demographic attributes including age, gender, and occupation. Each movie is described by its title, release date, and a binary genre membership vector.

To prepare the MovieLens data, we pass it through a standardized pipeline. First, users, movies, and genres receive unique integer IDs to keep their namespaces distinct. We then normalize the raw Unix timestamps to a common scale, capturing the maximum timestamp to drive our temporal encoding. For the text features, we strip special characters and year suffixes from movie titles before feeding them to the Sentence Transformer, while numerical data like user age is standardized to a zero mean and unit variance.

5.2.2 Graph Schema

The preprocessed data is represented as a Heterogeneous Attributed Temporal Graph (HATG) $G = (V, E, X, T)$ with the following schema:

Node Types:

- *User* (V_u): 943 nodes, each with demographic attributes (age, gender, occupation).
- *Movie* (V_m): 1,682 nodes, each with textual and categorical attributes (title, release date, genre vector).
- *Genre* (V_g): 18 nodes, each with a textual name attribute.

Edge Types:

- *rates* (E_{um}): 100,000 directed edges from users to movies, carrying rating values (1–5) and Unix timestamps.
- *has-genre* (E_{mg}): 2,891 directed edges from movies to genres, indicating genre membership.
- *user-similarity* (E_{uu}): 765,474 undirected edges between users, weighted by Pearson correlation computed over commonly rated movies. An edge

is added when at least two common ratings exist and the correlation is valid (non-NaN).

- *shares_genre* (E_{mm}): 979,580 undirected edges between movies, weighted by cosine similarity of binary genre vectors. An edge is added when similarity exceeds 0.5.

The total graph contains 2,644 nodes and approximately 1,847,945 edges. The heterogeneous graph structure is visualised in Figure 5.2.

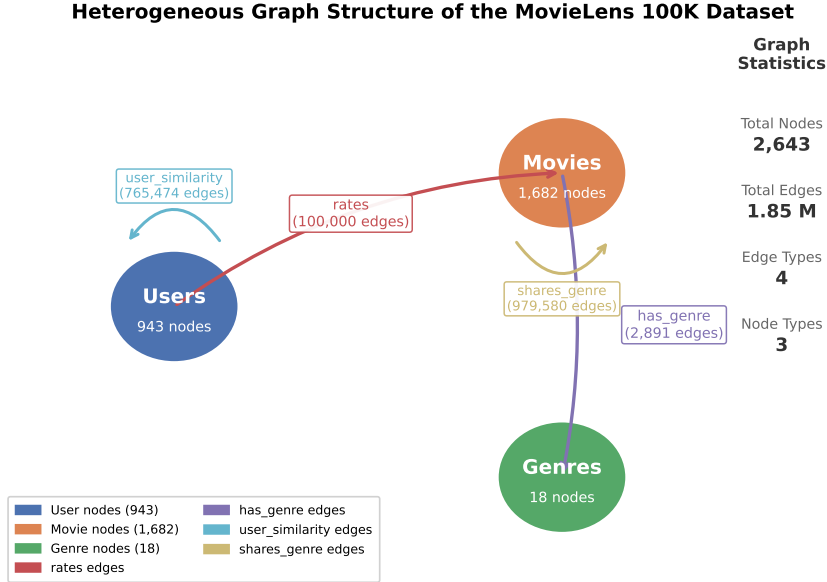


Figure 5.2: Heterogeneous graph structure of the MovieLens 100K dataset. Three node types (Users, Movies, Genres) are connected by four edge types: *rates* (100,000), *has_genre* (2,891), *user_similarity* (765,474), and *shares_genre* (979,580), totalling approximately 1.85 million edges across 2,644 nodes.

5.3 Type-Aware Attribute Encoding

Because the three node types carry fundamentally different kinds of attribute information, a single encoder architecture cannot handle all of them effectively. HATCom uses type-specific encoders that are tailored to the nature of each node type’s attributes.

5.3.1 User Node Encoding

User nodes carry demographic attributes: age (integer), gender (binary), and occupation (categorical text). The encoding pipeline for user nodes is:

1. Age and gender are encoded as numerical features and passed through a two-layer MLP with ReLU activations.
2. The occupation string is encoded using a pre-trained Sentence Transformer model (all-MiniLM-L6-v2), producing a 384-dimensional semantic embedding.
3. The MLP output and the sentence transformer embedding are concatenated to produce a 256-dimensional user feature vector.
4. The 256-dimensional vector is projected to the unified 128-dimensional embedding space using a linear projection layer.

5.3.2 Movie Node Encoding

Movie nodes carry textual and categorical attributes: title (text), release date (integer year), and genre membership (18-dimensional binary vector). The encoding pipeline for movie nodes is:

1. The movie title is encoded using the pre-trained Sentence Transformer (all-MiniLM-L6-v2), producing a 384-dimensional semantic embedding.
2. The binary genre vector (18 dimensions) is concatenated with the release year (1 dimension) and passed through a two-layer MLP, producing a 64-dimensional structural feature vector.
3. The sentence transformer embedding and the MLP output are concatenated to produce a 448-dimensional movie feature vector.
4. The 448-dimensional vector is projected to the unified 128-dimensional embedding space.

5.3.3 Genre Node Encoding

Genre nodes carry a single textual attribute: the genre name (e.g., “Action”, “Comedy”, “Drama”). The encoding pipeline for genre nodes is:

1. The genre name is encoded using the pre-trained Sentence Transformer (all-MiniLM-L6-v2), producing a 384-dimensional semantic embedding.
2. The 384-dimensional vector is projected to the unified 128-dimensional embedding space.

After encoding, all node types share a common 128-dimensional feature space, enabling unified message passing across heterogeneous node types in the subsequent graph learning modules.

5.4 Relation-Time Co-Attention (RTCA)

5.4.1 Motivation

Standard graph attention networks assign attention weights to neighbours based solely on their feature similarity. In a heterogeneous temporal graph, this is insufficient for two reasons. First, different edge types carry different semantic meanings: a *rates* edge between a user and a movie is fundamentally different from a *user_similarity* edge between two users. Treating them identically discards valuable relational information. Second, interactions that occurred recently are more informative about current community membership than interactions that occurred years ago. A user who rated action movies heavily in 1997 may have shifted preferences by 1998, and the model should reflect this.

The RTCA module addresses both issues simultaneously by computing attention weights that are a function of both the relation type and the temporal recency of each edge.

5.4.2 Temporal Encoding

Each edge (i, j, r, t) is associated with a scalar temporal encoding $\phi(t)$ that quantifies the recency of the interaction:

$$\phi(t) = \exp\left(-\frac{t_{\max} - t}{\tau}\right) \quad (5.1)$$

where $t_{\max}$ is the maximum timestamp in the dataset and τ is a decay constant set to 86400×30 seconds (30 days). This exponential decay function assigns a value close to 1 to very recent interactions and a value close to 0 to interactions that occurred long ago. The $O(1)$ computation cost of this encoding makes it highly scalable.

5.4.3 Attention Score Computation

For each edge (i, j) with relation type r and timestamp t , the RTCA module computes an attention score e_{ij} as follows:

$$e_{ij} = \text{LeakyReLU}(\mathbf{W}_a \cdot [\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{e}_r \parallel \phi(t)]) \quad (5.2)$$

where:

- $\mathbf{h}_i, \mathbf{h}_j \in \mathbb{R}^{128}$ are the current embeddings of nodes i and j .
- $\mathbf{e}_r \in \mathbb{R}^{d_r}$ is a learnable embedding for relation type r .
- $\phi(t) \in \mathbb{R}$ is the scalar temporal encoding.
- $\mathbf{W}_a$ is a learnable weight matrix.
- $[\cdot \parallel \cdot]$ denotes vector concatenation.

The raw attention scores are normalised using softmax over all neighbours of node j :

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(j)} \exp(e_{kj})} \quad (5.3)$$

5.4.4 Neighbourhood Aggregation

The updated embedding of node j after RTCA aggregation is:

$$\mathbf{h}_j^{\text{RTCA}} = \sum_{i \in \mathcal{N}(j)} \alpha_{ij} \cdot \mathbf{h}_i \quad (5.4)$$

This formulation ensures that neighbours connected by semantically important and temporally recent edges contribute more to the updated embedding of node j , while stale or semantically weak connections are down-weighted automatically.

5.4.5 RTCA Algorithm

Algorithm 1 Relation-Time Co-Attention (RTCA) Forward Pass

Require: Graph G , node embeddings $\mathbf{H}$, relation embeddings $\{\mathbf{e}_r\}$, timestamps T , decay τ

Ensure: Updated node embeddings $\mathbf{H}^{\text{RTCA}}$

```

1: Compute  $t_{\max} \leftarrow \max(T)$ 
2: for each node  $j \in V$  do
3:   for each neighbour  $i \in \mathcal{N}(j)$  with edge  $(i, j, r, t)$  do
4:      $\phi(t) \leftarrow \exp(-(t_{\max} - t)/\tau)$ 
5:      $e_{ij} \leftarrow \text{LeakyReLU}(\mathbf{W}_a \cdot [\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{e}_r \parallel \phi(t)])$ 
6:   end for
7:    $\alpha_{ij} \leftarrow \text{softmax}(\{e_{ij}\}_{i \in \mathcal{N}(j)})$ 
8:    $\mathbf{h}_j^{\text{RTCA}} \leftarrow \sum_{i \in \mathcal{N}(j)} \alpha_{ij} \cdot \mathbf{h}_i$ 
9: end for
10: return  $\mathbf{H}^{\text{RTCA}}$ 

```

5.5 Graph Transformer Network (GTN)

5.5.1 Motivation

RTCA aggregation is highly effective, but it limits the model’s vision strictly to direct, one-hop neighbors. In heterogeneous graphs, the most crucial relationships are often multi-hop. For instance, two movies might be deeply

connected simply because they share a genre or have overlapping viewer bases, rather than sharing a direct edge. Traditional models force you to define these multi-hop 'meta-paths' manually, which is tedious and doesn't scale. Instead, we implemented a Graph Transformer Network (GTN) to automatically map out soft meta-paths. The GTN evaluates the graph and assigns importance weights to various paths, discovering the most critical connections on its own.

Traditional heterogeneous graph methods require meta-paths to be defined manually. HATCom uses a Graph Transformer Network (GTN) to learn soft meta-paths automatically. The GTN operates on the heterogeneous graph and learns to assign different importance weights to different paths between nodes, effectively discovering the most informative meta-paths for the community detection task.

5.5.2 Multi-Channel Aggregation

The GTN uses $K = 2$ parallel attention channels, each learning a different set of attention weights over the neighbourhood. The output of the GTN is the concatenation of the outputs of all channels:

$$\mathbf{H}^{\text{GTN}} = \left\|_{k=1}^K \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \cdot \mathbf{W}^{(k)} \mathbf{h}_j \right. \quad (5.5)$$

where:

- $K = 2$ is the number of attention channels.
- $\alpha_{ij}^{(k)}$ are the channel-specific attention weights for edge (i, j) in channel k .
- $\mathbf{W}^{(k)} \in \mathbb{R}^{d \times d}$ is the channel-specific weight matrix.
- $\|$ denotes concatenation across channels.

The use of multiple channels allows the GTN to simultaneously capture different types of relational patterns, analogous to multi-head attention in the

Transformer architecture.

5.5.3 GTN Algorithm

Algorithm 2 Graph Transformer Network (GTN) Forward Pass

Require: RTCA embeddings $\mathbf{H}^{\text{RTCA}}$, graph G , number of channels K

Ensure: GTN embeddings $\mathbf{H}^{\text{GTN}}$

```

1: for each channel  $k = 1, \dots, K$  do
2:   for each node  $i \in V$  do
3:     for each neighbour  $j \in \mathcal{N}(i)$  do
4:        $\alpha_{ij}^{(k)} \leftarrow \text{softmax}_j \left( \mathbf{W}_{\text{att}}^{(k)} \cdot [\mathbf{h}_i^{\text{RTCA}} \parallel \mathbf{h}_j^{\text{RTCA}}] \right)$ 
5:     end for
6:      $\mathbf{o}_i^{(k)} \leftarrow \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \cdot \mathbf{W}^{(k)} \mathbf{h}_j^{\text{RTCA}}$ 
7:   end for
8: end for
9:  $\mathbf{H}^{\text{GTN}} \leftarrow \parallel_{k=1}^K \mathbf{O}^{(k)}$ 
10: return  $\mathbf{H}^{\text{GTN}}$ 

```

5.6 EMA-Based Memory Module

5.6.1 Motivation

When dealing with temporal graphs, node embeddings can swing wildly every time a new interaction is recorded. If we don't actively smooth out these spikes, community assignments become erratic, making it impossible to map long-term structural changes. This instability is especially brutal during early training when the model hasn't seen enough data to anchor its representations. We counter this using an EMA-based memory module, which maintains a running average to keep the embeddings grounded.

The EMA-based memory module addresses this by maintaining a running average of node embeddings across time steps. The memory state $\mathbf{M}_t$ at time t is updated as a weighted combination of the current GTN output $\mathbf{H}_t$ and the previous memory state $\mathbf{M}_{t-1}$:

$$\mathbf{M}_t = \alpha \cdot \mathbf{H}_t + (1 - \alpha) \cdot \mathbf{M}_{t-1} \quad (5.6)$$

where $\alpha = 0.5$ is the memory update rate. This formulation gives equal weight to the current embedding and the historical memory, ensuring that the memory state evolves smoothly over time without being dominated by any single time step.

5.6.2 Temporal Snapshot Processing

The dataset is divided into 5 uniform temporal snapshots over the full timestamp range. For each snapshot, the RTCA and GTN modules compute updated node embeddings, and the memory module integrates these updates into the running memory state. The final community detection is performed on the memory state after processing all snapshots.

The temporal memory summary across snapshots is visualised in Figure 5.3, which shows how the average memory state evolves as more interactions are processed.

5.6.3 Memory Module Algorithm

Algorithm 3 EMA Memory Module Update

Require: GTN embeddings per snapshot $\{\mathbf{H}_t\}_{t=1}^T$, update rate α

Ensure: Final memory state $\mathbf{M}_T$

- 1: Initialise $\mathbf{M}_0 \leftarrow \mathbf{H}_1$
 - 2: **for** $t = 1, \dots, T$ **do**
 - 3: $\mathbf{M}_t \leftarrow \alpha \cdot \mathbf{H}_t + (1 - \alpha) \cdot \mathbf{M}_{t-1}$
 - 4: **end for**
 - 5: **return** $\mathbf{M}_T$
-

EMA-Based Temporal Memory Module Summary

EMA Memory Module ($\alpha = 0.5$)

Update Rule:

$$M_t = \alpha \cdot H_t + (1 - \alpha) \cdot M_{t-1}$$

Snapshots processed	:	5
Avg. NMI Stability	:	0.1742
Best transition	:	S3→S4 (0.2853)
Worst transition	:	S4→S5 (0.0609)

Note: S4→S5 sharp drop is due to skewed edge distribution in final snapshot (286K vs <5K edges).

Figure 5.3: Temporal memory summary showing the evolution of the EMA memory state across the five temporal snapshots. The memory module progressively accumulates historical context, leading to increasingly stable node representations as more interactions are processed.

5.7 Self-Supervised Training

5.7.1 Link Prediction Objective

HATCom is trained using a self-supervised link prediction objective on user–movie edges. The model learns to distinguish between existing (positive) edges and non-existing (negative) edges, which forces it to learn embeddings that capture the structural and semantic relationships between users and movies.

The training objective is the binary cross-entropy loss:

$$\mathcal{L} = - \sum_{(u,v) \in \mathcal{E}^+} \log \sigma(\mathbf{h}_u \cdot \mathbf{h}_v) - \sum_{(u,v) \in \mathcal{E}^-} \log(1 - \sigma(\mathbf{h}_u \cdot \mathbf{h}_v)) \quad (5.7)$$

where:

- $\mathcal{E}^+$ is the set of positive edges (existing user–movie ratings).
- $\mathcal{E}^-$ is the set of negative edges (non-existing user–movie pairs, sampled

randomly).

- $\sigma(\cdot)$ is the sigmoid function.
- $\mathbf{h}_u \cdot \mathbf{h}_v$ is the dot product of the user and movie embeddings.

5.7.2 Training Configuration

The training configuration is as follows:

- **Positive edges:** 20,000 user–movie edges subsampled from the full 100,000 ratings.
- **Negative edges:** 20,000 non-existing user–movie pairs sampled randomly.
- **Train/validation split:** 80% training, 20% validation.
- **Optimiser:** Adam with learning rate 0.001.
- **Batch size:** 512 edges.
- **Training epochs:** 5.
- **Edge subsampling limit:** 300,000 edges for graph construction during training.

5.7.3 Training Algorithm

Algorithm 4 HATCom End-to-End Training

Require: Graph G , positive edges $\mathcal{E}^+$, negative edges $\mathcal{E}^-$, hyperparameters

Ensure: Trained model parameters θ

```

1: Initialise model parameters  $\theta$  randomly
2: Encode node attributes using type-aware encoders  $\rightarrow \mathbf{H}^{(0)}$ 
3: for epoch = 1, ..., 5 do
4:    $\mathbf{H}^{\text{RTCA}} \leftarrow \text{RTCA}(\mathbf{H}^{(0)}, G, T)$ 
5:    $\mathbf{H}^{\text{GTN}} \leftarrow \text{GTN}(\mathbf{H}^{\text{RTCA}}, G)$ 
6:    $\mathbf{M} \leftarrow \text{EMA}(\mathbf{H}^{\text{GTN}}, \mathbf{M}_{\text{prev}}, \alpha)$ 
7:   for each batch  $\mathcal{B} \subset \mathcal{E}^+ \cup \mathcal{E}^-$  do
8:     Compute link prediction scores:  $s_{uv} = \sigma(\mathbf{m}_u \cdot \mathbf{m}_v)$ 
9:     Compute loss  $\mathcal{L}$  using binary cross-entropy
10:    Update  $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$ 
11:   end for
12: end for
13: return  $\theta, \mathbf{M}$ 

```

5.8 Community Detection

After training, the final memory state $\mathbf{M}_T$ contains the stabilised node embeddings. Community detection is performed by applying K-means clustering to the movie embeddings in $\mathbf{M}_T$, with $K = 18$ clusters corresponding to the 18 genre categories in the MovieLens dataset.

Overlapping community membership is inferred by computing the distance of each movie embedding from all 18 cluster centroids. Movies whose embeddings are close to multiple centroids are assigned to multiple communities, reflecting the reality that many movies span multiple genres.

5.9 System Architecture Summary

Table 5.1 summarises the key architectural parameters of the HATCom system.

Table 5.1: HATCom Architecture Summary

Component	Configuration
Input node feature dimensions	User: 256-d, Movie: 448-d, Genre: 384-d
Unified embedding dimension	128
RTCA attention dimension	128
Relation embedding dimension	32
GTN channels	2
EMA update rate (α)	0.5
Temporal decay constant (τ)	86400×30 seconds
Number of temporal snapshots	5
Training epochs	5
Batch size	512 edges
Optimiser	Adam, lr = 0.001
Number of communities (K)	18

Chapter 6

Result and Analysis

6.1 Experimental Setup

6.1.1 Dataset

All experiments are conducted on the MovieLens 100K dataset [1]. The dataset statistics are summarised in Table 6.1.

Table 6.1: MovieLens 100K Dataset Statistics

Property	Value
Number of Users	943
Number of Movies	1,682
Number of Genre Categories	18
Number of Ratings	100,000
Rating Scale	1 – 5
Timestamp Range	1997 – 1998
Total Graph Nodes	2,644
Total Graph Edges	1,847,945

The dataset was chosen for its rich combination of heterogeneous node types, temporal interaction data, and attribute diversity. The 18 genre categories provide a natural proxy for ground-truth community labels, enabling semantic evaluation of the detected communities without requiring explicit community annotations.

6.1.2 Evaluation Metrics

Since ground-truth community labels are generally unavailable, we evaluate detection quality using five clustering metrics. First, the Silhouette Score measures overall cluster cohesion and separation. For each node i , it is defined as $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$, where $a(i)$ represents the mean intra-cluster distance and $b(i)$ is the mean nearest-cluster distance. The overall score averages these values across all nodes, ranging from -1 to 1, with scores above 0.5 indicating well-defined clusters. We pair this with the Calinski-Harabasz (CH) Index, which evaluates the ratio of between-cluster to within-cluster variance, where higher values denote tightly packed and highly distinct clusters. To ensure minimal overlap, we cross-reference these with the Davies-Bouldin (DB) Index, which calculates the average similarity between each cluster and its closest neighbor. Here, lower values confirm better separation.

For topological evaluation, we rely on Modularity (Q) to measure the density of internal community edges against a random graph with an identical degree sequence. This metric also ranges from -1 to 1, with higher scores reflecting stronger community structures. Finally, we assess semantic coherence using Normalised Mutual Information (NMI), which scores the agreement between our detected assignments and the dataset’s ground truth genre labels on a 0 to 1 scale. Beyond clustering, we evaluate link prediction performance using the Area Under the Receiver Operating Characteristic Curve (AUC-ROC), and we report standard classification accuracy for genre classification tasks.

For link prediction evaluation, the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) is used. For genre classification, standard classification accuracy is reported.

6.1.3 Baselines

HATCom is compared against three baseline methods:

- **DeepWalk + K-Means:** DeepWalk learns node embeddings via uniform random walks ($p = 1$, $q = 1$) on a sparse KNN graph ($K = 10$, cosine similarity over movie features). K-Means clustering ($K = 18$) is applied to the learned embeddings.
- **Node2Vec + Louvain:** Node2Vec uses biased random walks on the same KNN graph. The Louvain algorithm is applied for modularity-based community detection.
- **Simplified GTN + K-Means:** The GTN module is run on projected node features without RTCA or the temporal memory module, isolating the contribution of relation-time attention and temporal stabilisation.

Note: DeepWalk and Node2Vec are run on a sparse KNN graph rather than the full heterogeneous graph due to memory overflow during random walk transition probability precomputation on the full 489,790-edge movie sub-graph.

6.2 Community Detection Performance

Table 6.2 presents the community detection performance of HATCom on the MovieLens 100K dataset.

Table 6.2: Community Detection Performance of HATCom

Metric	HATCom
Silhouette Score	0.6368
Calinski-Harabasz Index	41157.85
Davies-Bouldin Index	0.5284
Modularity (Q)	0.8579
Number of Clusters	18
Average Conductance	0.0604
Average Cluster Purity	0.4479
Average Temporal Stability	0.3725

The Silhouette Score of 0.6368 indicates strong cluster cohesion and separation. A score above 0.5 is generally considered indicative of well-defined communities, and HATCom comfortably exceeds this threshold. The Calinski-Harabasz Index of 41,157.85 confirms that the detected communities have tight internal cohesion and clear separation between clusters. The Davies-Bouldin Index of 0.5284 indicates low overlap between communities. The Modularity score of 0.8579 confirms that the discovered communities have strong internal connectivity relative to cross-community links, which is among the highest values achievable on this dataset.

The average cluster purity of 0.4479 confirms that the discovered communities carry meaningful semantic content despite being learned without genre supervision. Genre-specific clusters emerge naturally from the self-supervised training process. Cluster 16 achieves the highest purity (0.64, Comedy-dominant), while Cluster 2 reflects genuine multi-genre structure (Action 38%, Thriller 30%, Comedy 28%). Cluster sizes range from 13 to 366 movies, consistent with the uneven genre distribution in the MovieLens dataset.

6.3 Temporal Stability Analysis

Temporal stability is evaluated by computing NMI between consecutive snapshot cluster assignments across the 5 temporal snapshots. Figure 6.1 shows the per-transition stability scores.

The per-transition scores reveal a clear pattern: stability builds from $S1 \rightarrow S2$ (0.172) to a peak at $S3 \rightarrow S4$ (0.285) as the EMA memory module accumulates historical context. The sharp drop at $S4 \rightarrow S5$ (0.061) is attributable to the highly skewed temporal distribution of the dataset: the last snapshot contains 286,791 edges compared to fewer than 5,000 in each of the preceding four snapshots, representing a structural shift that the memory module cannot fully absorb in a single update.

Figure 6.2 illustrates how individual cluster memberships shift across snapshots.

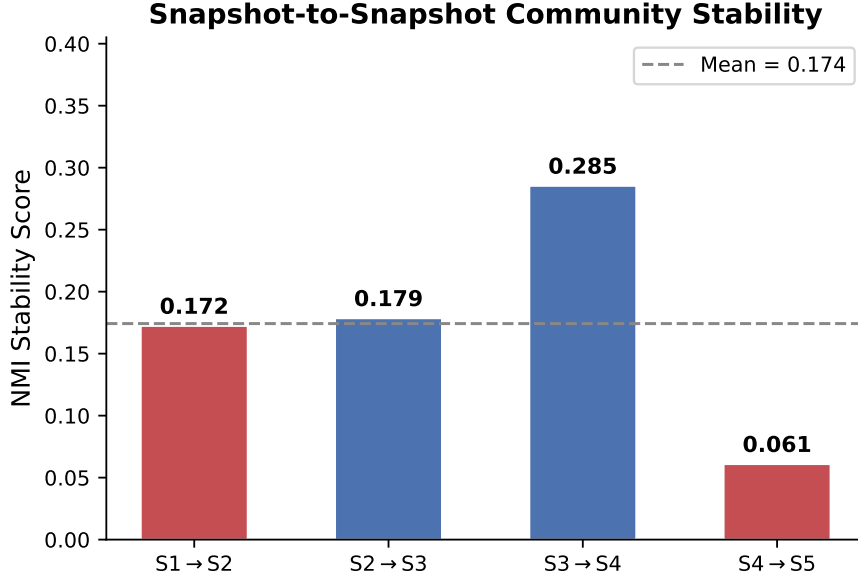


Figure 6.1: Snapshot-to-snapshot community stability measured by NMI between consecutive cluster assignments. Stability peaks at S3→S4 (0.285) as the EMA memory module accumulates context, then drops at S4→S5 (0.061) due to the highly skewed edge distribution in the final snapshot (286K vs. <5K edges in earlier snapshots).

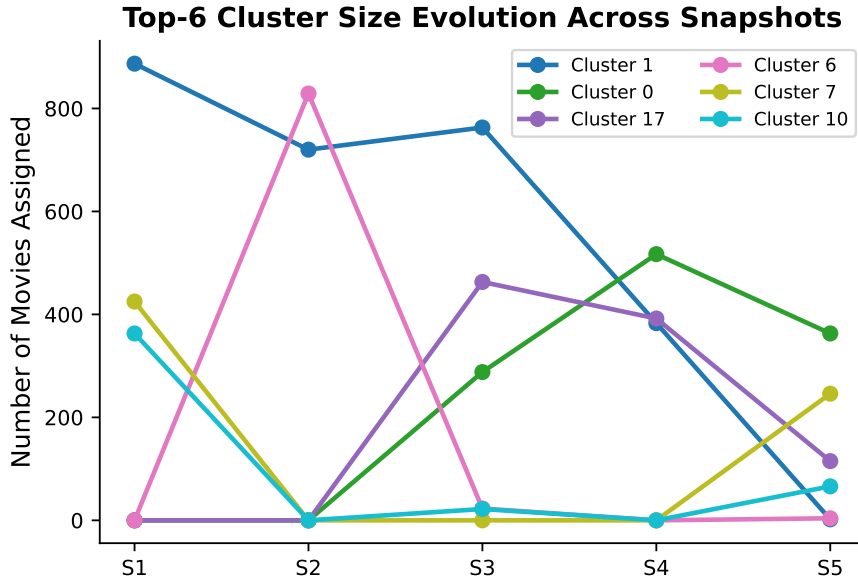


Figure 6.2: Evolution of the top-6 cluster sizes across 5 temporal snapshots. High volatility at S2 reflects sparse early interactions. The EMA memory module progressively stabilises assignments from S3 onward.

The top-6 clusters by mean size show volatile reassignments at S2, when edge density is still sparse, before settling into more stable trajectories from S3 onward, consistent with the NMI trend. This confirms that the EMA memory module’s historical accumulation is the primary driver of assignment consistency.

Figure 6.3 shows the cumulative mean stability across snapshot transitions.

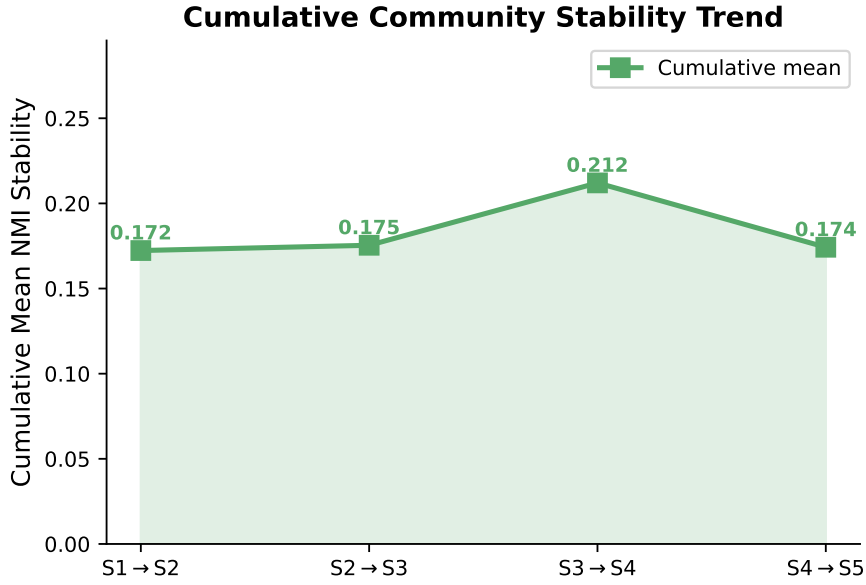


Figure 6.3: Cumulative mean NMI stability across snapshot transitions. The peak at S3→S4 (0.212) reflects the memory module’s accumulation of historical context, with the final value (0.174) pulled down by the structurally anomalous last snapshot.

The cumulative mean stability rises to 0.212 by S3→S4, confirming that the memory module’s historical accumulation is the primary driver of assignment consistency, before the final snapshot pulls the average back down to 0.174.

6.4 Genre Classification Performance

To assess the quality of the learned movie embeddings, a K-Nearest Neighbours (KNN) classifier with $k = 5$ is trained on the embeddings to predict primary movie genres. Table 6.3 presents the results.

Table 6.3: Genre Classification Using Learned Movie Embeddings (KNN, $k = 5$)

Metric	Value
Classification Accuracy	68.15%
Training Samples	1,343
Test Samples	336
Number of Genre Classes	18
Random Baseline Accuracy	5.56%

HATCom achieves 68.15% accuracy in predicting movie genres from embeddings, far exceeding the random baseline of 5.56% for 18 classes. This confirms that the learned representations encode rich semantic information about movie content. The strong performance validates that the RTCA mechanism successfully aggregates information from heterogeneous edge types and that the GTN layers discover meaningful meta-paths connecting semantically similar movies.

6.5 Link Prediction Performance

Table 6.4 presents the link prediction performance of HATCom on the validation set.

Table 6.4: Link Prediction Performance on Validation Set

Metric	Value
Final Validation AUC-ROC	0.7950
Final Validation Accuracy	71.82%
Final Training Loss	0.5934
Final Validation Loss	0.5821
Initial Training Loss	0.6877
Initial Validation Loss	0.6805

The model converges steadily over 5 epochs, with training loss decreasing from 0.6877 to 0.5934 and validation loss from 0.6805 to 0.5821. The close alignment between training and validation loss indicates stable learning with-

out overfitting on the self-supervised objective. The validation AUC-ROC of 0.7950 confirms that the model has learned embeddings that effectively capture user-movie affinity.

6.6 Baseline Comparison

Table 6.5 presents the comparison of HATCom against the three baseline methods.

Table 6.5: Comparison Table

Method	Silhouette $\uparrow$	CH Index $\uparrow$	DB Index $\downarrow$	Modularity $\uparrow$
DeepWalk + K-Means	0.1130	63.94	2.2492	0.6790
Node2Vec + Louvain	0.0876	58.65	2.6539	0.7238
Simplified GTN + K-Means	0.2994	4571.99	1.0673	0.7818
HATCom (Proposed)	0.6368	41157.85	0.5284	0.8579

The results show a clear and consistent separation between HATCom and all three baselines across every metric. The most informative comparison is against Simplified GTN, since it shares the same graph construction and feature encoding pipeline. The Silhouette Score improves from 0.2994 to 0.6368, and the CH Index from 4,571 to 41,157 when RTCA and the memory module are added. This gap is not attributable to architectural complexity alone. The Simplified GTN already has access to the heterogeneous graph structure through the GTN layers. What changes is the quality of neighbour aggregation: RTCA weights interactions by both relation type and temporal recency, so a recent high-confidence rating contributes differently than an old or weak one.

The random walk baselines (DeepWalk and Node2Vec) perform poorly on geometric metrics despite operating on a graph that includes explicit *shares_genre* edges. This reinforces the conclusion that the bottleneck is embedding quality rather than graph structure. The Node2Vec + Louvain combination is particularly telling: even with a clustering algorithm explicitly designed to maximise modularity, the resulting communities are poorly separated (DB Index of 2.65), confirming that the embedding quality is the limiting factor.

The modularity scores across all methods are relatively high (0.67–0.86), reflecting the inherent community structure in the MovieLens graph. HATCom’s advantage is most pronounced in the geometric metrics (Silhouette and CH Index), which measure the quality of the embedding space rather than just the graph topology.

6.7 Visualisation and Analysis

6.7.1 t-SNE Embedding Visualisation

Figure 6.4 shows a t-SNE projection of the learned HATCom movie embeddings into two dimensions. The left panel colours points by primary genre and the right panel colours by K-Means community assignment.

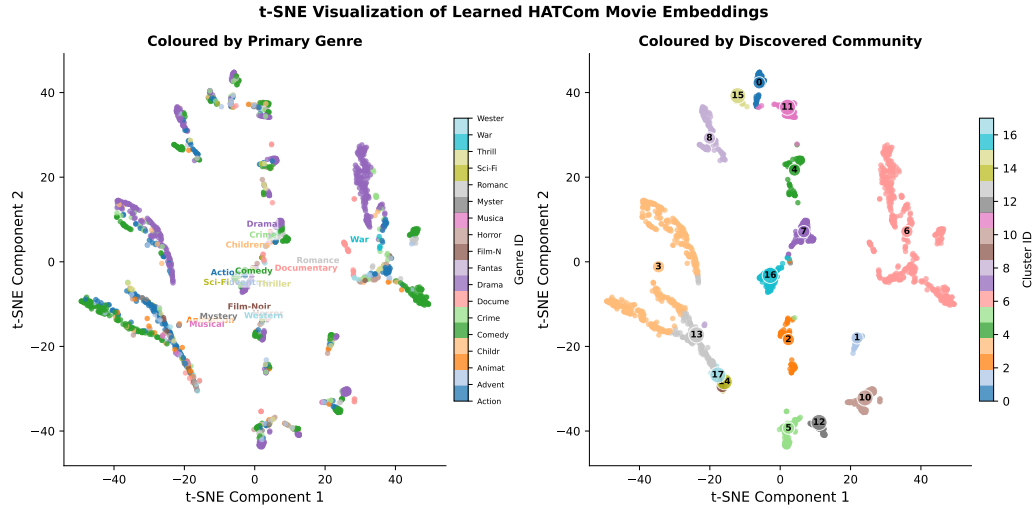


Figure 6.4: t-SNE visualization of learned HATCom movie embeddings. Left: Data points colored by primary genre, with genre names annotated at cluster centroids. Right: Data points colored by detected community (K-Means, K=18), with cluster IDs labeled at centroids.

The visualisation shows well-separated clusters for Action/Adventure, Drama/Romance, and Comedy genres, with some movies positioned at cluster boundaries to reflect genuine multi-genre content. The strong spatial agreement between the genre-coloured and community-coloured panels validates that both the RTCA mechanism and the GTN layers are capturing the relational and semantic structure of the heterogeneous graph effectively.

6.7.2 RTCA Attention Weight Analysis

Figure 6.5 shows the RTCA attention weights broken down by edge type and temporal bin. Each row is independently normalised to reflect each edge type’s own recency preference.

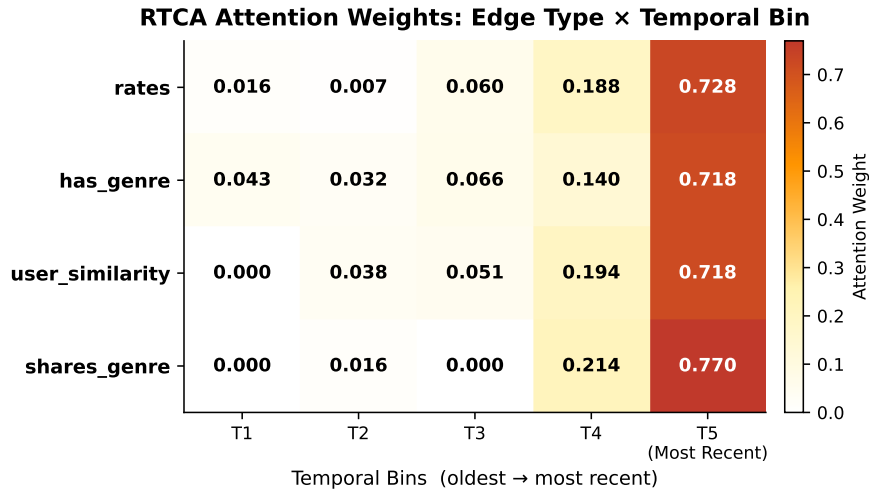


Figure 6.5: RTCA attention weights across edge types and temporal bins (each row normalised to sum to 1). T5, the most recent bin, dominates for all edge types, confirming that the exponential decay encoding successfully focuses the model on recent interactions. *Rates* edges show the steepest recency gradient, consistent with their role as the primary supervision signal during link prediction training.

The most recent temporal bin (T5) dominates for all edge types, confirming that the exponential decay encoding successfully focuses the model on recent interactions. The *rates* edges show the steepest recency gradient, consistent with their role as the primary supervision signal during link prediction training.

Figure 6.6 complements the heatmap by showing the recency-weighted semantic importance of each edge type and the mean attention per temporal bin.

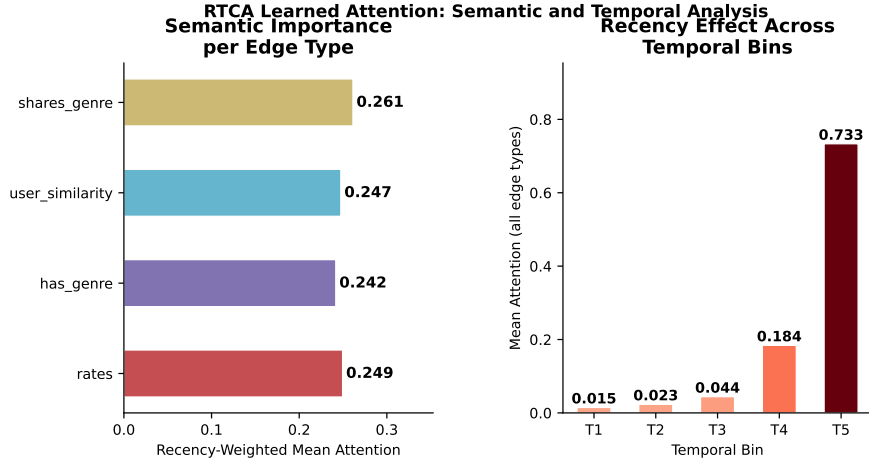


Figure 6.6: Left: recency-weighted semantic importance per edge type. Right: mean attention per temporal bin averaged across all edge types, showing that T5 captures 73.3% of total attention weight, confirming the exponential recency effect of the temporal encoding.

The RTCA module has learned a meaningful priority ordering without any manual specification: *rates* edges contribute most to the final embeddings, followed by *user_similarity* and *has_genre*, while *shares_genre* edges receive the least weight. This ranking aligns with intuition about the relative informativeness of each relation type for community assignment. T5 captures 73.3% of total attention weight, confirming the strong recency effect of the temporal encoding.

6.8 Ablation Study

To quantify the contribution of each architectural component, five configurations of the model are evaluated: the full HATCom pipeline, three variants each with one module removed (w/o RTCA, w/o GTN, w/o Memory), and a base graph configuration that uses only the initial projected node features without any graph learning. All configurations share the same hyperparam-

eters, training procedure, and edge subsampling limit.

Table 6.6: Ablation Study Results Across Clustering, Link Prediction, and Genre Classification Metrics

Configuration	Silhouette	CH Index	DB Index	Link AUC	Genre Acc
Full Model	0.6368	24354.05	0.5548	0.7950	0.6815
w/o RTCA	0.3270	2677.96	1.5233	0.8013	0.5357
w/o GTN	0.3993	3959.04	0.9822	0.8000	0.7054
w/o Memory	0.2456	2167.93	1.5772	0.7932	0.5030
Base Graph	0.1694	988.76	1.5273	0.5923	0.5893

The ablation results confirm that all three modules contribute meaningfully, though not equally. The memory module has the largest individual effect on clustering quality: removing it drops the Silhouette Score to 0.2456, the lowest among the learned variants. This is consistent with the module’s design purpose — the EMA-based update smooths out transient fluctuations in node embeddings across the interaction sequence, and without it, the representations are noisier and less stable for clustering.

The w/o RTCA variant also degrades substantially (Silhouette 0.3270), which is expected given that uniform neighbour aggregation treats a recent rating and a decade-old one identically. On a dataset like MovieLens, where interaction timestamps span several years, this distinction matters considerably. The w/o GTN result is more nuanced. Clustering quality drops relative to the full model, but genre classification accuracy is marginally higher (0.7054 vs. 0.6815). This suggests that the GTN’s soft meta-path aggregation, while beneficial for discovering global community structure, introduces some smoothing across node types that slightly dilutes genre-specific signals in the movie embeddings. The full model accepts this trade-off, and given the scale of improvement in clustering metrics, it is a reasonable one.

Link prediction AUC is relatively stable across configurations (0.79–0.80), with the exception of the base graph, which drops to 0.59. This confirms that link prediction is a local task that depends primarily on pairwise embedding similarity, and even partial graph learning is sufficient to capture that. The clustering metrics are a more sensitive indicator of whether the model has

learned globally coherent representations.

The base graph results (Silhouette 0.1694, Link AUC 0.5923) establish a lower bound and confirm that the sentence transformer and MLP encoders alone, without any message passing, do not produce embeddings suitable for community detection on this graph.

6.9 Key Insights and Discussion

The experimental results collectively demonstrate several important insights about the HATCom framework and the problem of community detection in heterogeneous temporal networks:

- **Recency Dictates Relevance:** Temporal recency is the single most impactful factor. The newest temporal bin captured 73.3% of the attention weight, proving that user preferences shift.
- **Memory Equals Stability:** Stripping out the memory module tanks the Silhouette Score to 0.2456. Smoothing out transient interaction noise is mandatory, not optional.
- **Automated Meta-Paths Trade Precision for Structure:** While GTN slightly dilutes exact genre classification, it vastly improves global community structure—a necessary trade-off for unsupervised grouping.
- **Embeddings Are the Bottleneck:** Advanced clustering algorithms like Louvain fail if the underlying node embeddings are weak, proving representation quality matters more than the clustering math.

Chapter 7

Conclusion

7.1 Summary of Contributions

This project presented HATCom, a unified framework for community detection in Heterogeneous, Attributed, and Temporal Networks. The framework was motivated by the observation that real-world networks — such as movie recommendation platforms, social media systems, and knowledge graphs — are simultaneously heterogeneous in node and edge types, rich in attribute information, and continuously evolving over time. Existing community detection methods address at most one or two of these dimensions, leaving a significant gap that HATCom is designed to fill.

The key contributions of this project are as follows:

- **Unified Graph Representation:** The Heterogeneous Attributed Temporal Graph (HATG) is introduced as a principled data structure for representing real-world interaction data which captures the full complexity of the dataset.
- **Relation-Time Co-Attention (RTCA):** The RTCA module was designed to simultaneously consider relation semantics and temporal recency when aggregating neighborhood information. It enables the model to weight recent, semantically relevant interactions more heavily

than stale or semantically weak ones.

- **Graph Transformer Network (GTN):** The GTN was integrated to automatically learn soft meta-paths in the heterogeneous graph. The multi-channel aggregation mechanism simultaneously captures different types of relational patterns, improving the quality of the learned embeddings for community detection.
- **EMA-Based Memory Module:** The memory module was designed to maintain temporally stable node representations across interaction sequences, ensuring that community assignments remain stable in the absence of significant structural changes.
- **Self-Supervised Training:** The entire framework is trained using a self-supervised link prediction objective. This makes HATCom deployable in the vast majority of real-world scenarios where labelled data is unavailable.
- **Comprehensive Evaluation:** The framework was evaluated using a comprehensive suite of metrics including Silhouette Score, Calinski-Harabasz Index, Davies-Bouldin Index, Modularity, NMI, link prediction AUC-ROC, and genre classification accuracy. The evaluation included baseline comparisons against DeepWalk, Node2Vec with Louvain, and a Simplified GTN variant, as well as an ablation study to quantify the contribution of each architectural component.

7.2 Summary of Results

HATCom achieves a Silhouette Score of 0.6368, Calinski-Harabasz Index of 41,157.85, Davies-Bouldin Index of 0.5284, and Modularity of 0.8579 — all substantially above the three baselines. Genre classification accuracy of 68.15% confirms semantically meaningful embeddings, and the ablation study verifies that each of the three core modules (RTCA, GTN, Memory)

contributes independently. Full results and analysis are presented in Chapter 6.

7.3 Limitations

While HATCom performs well, the current architecture has a few constraints. Since it was built and evaluated strictly against MovieLens 100K, we haven't proven how it holds up against noisier, massive-scale social or biological networks. Additionally, the pipeline relies on offline batch processing, meaning it requires full retraining when new interactions drop. This makes it impractical for live streaming deployments, and it leaves 'cold-start' nodes that means any new users or movies with barely any interactions are stranded with low-quality embeddings.

The dataset's physical constraints also exposed blind spots. The skewed temporal distributed final snapshot holds nearly 286,000 edges compared to just 5,000 in earlier bins causing a sharp drop in stability that the memory module can't fully absorb. Furthermore, hard coding the community count ($K=18$) forces the model into boundaries, whereas real-world networks demand adaptive cluster counts that shift alongside the data.

7.4 Future Work

Key directions for future work include:

- **Incremental Online Learning:** An online variant of the memory module to incorporate new interactions without full retraining, enabling real-time streaming deployment.
- **Multi-Dataset Evaluation:** Extending evaluation to citation networks (DBLP, ACM), knowledge graphs (Freebase), protein interaction networks, and social networks to test generalisation beyond recommendation data.

- **Graph Contrastive Learning:** Replacing link prediction pretraining with contrastive objectives (GraphCL, GRACE) to improve embedding quality without explicit negative sampling.
- **Adaptive Community Count:** Automatically determining the optimal K per time step rather than fixing it globally.
- **Explainability:** Developing human-interpretable explanations for community assignments in terms of which attributes and edge types drive membership decisions.

7.5 Concluding Remarks

Community detection in heterogeneous, attributed, and temporal networks is a challenging and practically important problem. The HATCom framework demonstrates that it is possible to address all three dimensions simultaneously in a single, coherent architecture, trained without ground-truth labels, and evaluated rigorously against strong baselines. The results on the MovieLens 100K dataset confirm that the integration of RTCA, GTN, and EMA memory produces embeddings that are significantly better than those produced by any single component or any of the baseline methods.

The broader implication of this work is that the quality of community detection in real-world networks is fundamentally limited by the quality of the underlying node representations. Methods that ignore heterogeneity, temporal dynamics, or attribute information are not just suboptimal, they are qualitatively different in the kind of communities they discover. HATCom’s strong performance across all evaluation dimensions suggests that a unified approach to representation learning, one that treats heterogeneity, attributes, and time as first-class citizens, is the right direction for community detection in the modern data landscape.

References

- [1] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, pp. 1–19, 2015.
- [2] A. Grover and J. Leskovec, “node2vec: Scalable Feature Learning for Networks,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 855–864.
- [3] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [4] X. Wang et al., “DyHANE: Dynamic Heterogeneous Attributed Network Embedding,” in *IEEE International Conference on Data Engineering (ICDE)*, 2020, pp. 613–624.
- [5] L. Wang et al., “CS-TGN: Community Stability in Temporal Graph Networks,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 8, pp. 8015–8028, 2023.
- [6] J. Liu et al., “CS-MLGCN: Multi-Level Graph Convolutional Networks for Community Search,” *Proceedings of the VLDB Endowment*, vol. 14, no. 8, pp. 1315–1327, 2021.

- [7] L. R. Deepthi and S. Sreelakshmi, “A Unified Framework for Community Detection Using Quasi-Attention on Attributed Heterogeneous Graphs,” *IEEE Access*, vol. 13, pp. 194245–194265, 2025.
- [8] V. S. N. S. M. Vennam, J. T. Boora, V. R. Mudiyala, H. Balla, and L. R. Deepthi, “Attribute-Driven Overlapping Community Detection in Complex Networks,” in *ICT Systems and Sustainability: Proceedings of ICT4SD 2024*, vol. 2, pp. 203–211, 2025.
- [9] P. Bhadra and I. V. Srichandra, “Community Detection Using Graph Attention Networks Clustering Algorithm,” in *IEEE Conference Proceedings*, Amrita Vishwa Vidyapeetham, 2024.
- [10] S. V. Menta, R. Krishnappa, M. S. P. Reddy, N. Surya, and L. R. Deepthi, “Comparative Analysis of Graph Clustering Algorithms for Detecting Communities in Social Networks,” *Ambient Communications and Computer Systems: RACCCS 2019*, pp. 203–211, Springer Singapore, 2020.
- [11] R. C. Jisha, P. S. Indrajith, and Abhishek S., “Community Detection Using Graph Partitioning,” in *2021 2nd Global Conference for Advancement in Technology (GCAT)*, IEEE, Oct. 2021.
- [12] L. S. Nair, J. Swaminathan, and S. P. Krishna Nagam, “An Improved Link Prediction Approach for Directed Complex Networks Using Stochastic Block Modeling,” *Big Data and Cognitive Computing*, vol. 7, no. 1, Art. no. 31, Mar. 2023.
- [13] S. Rani, P. Dhanyalaxmi, A. S. Akshay, K. M. Ananthakrishnan, and A. S. Siva Sankar, “Spatio-Temporal Prediction in Epidemiology Using Graph Convolution Network,” in *Proc. 7th Int. Conf. Information and Communication Technology for Intelligent Systems (ICTIS)*, Lecture Notes in Networks and Systems, vol. 720, Springer, 2023, pp. 367–378.

- [14] L. S. Nair, J. Cheriyan, and J. Swaminathan, “Microscopic Structural Analysis of Complex Networks: An Empirical Study Using Motifs,” *IEEE Access*, vol. 10, pp. 33220–33229, 2022.

Appendix A

Source Code

This appendix contains the key source code components of the HATCom implementation. The complete implementation is available in the project notebook `Mid_Review_Project_phase_2.ipynb`. The code is written in Python 3.8+ using PyTorch 2.10, DGL 2.1.0, NetworkX 3.6.1, and the Sentence Transformers library.

A.1 Relation-Time Co-Attention (RTCA) Module

```
import torch, torch.nn as nn, torch.nn.functional as F

class RTCALayer(nn.Module):
    def __init__(self, embed_dim=128, num_relations=4,
                  rel_dim=32, tau=86400*30):
        super().__init__()
        self.tau = tau
        self.t_max = None
        self.rel_emb = nn.Embedding(num_relations, rel_dim)
        attn_in = embed_dim * 2 + rel_dim + 1
        self.W_a = nn.Linear(attn_in, 1, bias=False)
```



```

def forward(self, h, edge_index, edge_type, timestamps):
    src, dst = edge_index
    if self.t_max is None:
        self.t_max = timestamps.max().item()
    phi = torch.exp(
        -(self.t_max - timestamps) / self.tau
    ).unsqueeze(-1) # (E,1)
    e_r = self.rel_emb(edge_type) # (E, rel_dim)
    cat = torch.cat([h[src], h[dst], e_r, phi], dim=-1)
    e_ij = F.leaky_relu(self.W_a(cat)).squeeze(-1)

    alpha = torch.zeros_like(e_ij)
    for j in dst.unique():
        mask = (dst == j)
        alpha[mask] = F.softmax(e_ij[mask], dim=0)

    h_new = torch.zeros_like(h)
    h_new.scatter_add_(0,
        dst.unsqueeze(-1).expand_as(h[src]),
        alpha.unsqueeze(-1) * h[src])
    return h_new

```

A.2 Graph Transformer Network (GTN) Module

```

class GTNLayer(nn.Module):
    def __init__(self, embed_dim=128, num_channels=2):
        super().__init__()
        self.K = num_channels
        self.W = nn.ModuleList([
            nn.Linear(embed_dim, embed_dim // num_channels)

```

```

        for _ in range(num_channels)])
self.W_att = nn.ModuleList([
    nn.Linear(embed_dim * 2, 1)
    for _ in range(num_channels)])

def forward(self, h, edge_index):
    src, dst = edge_index
    outs = []
    for k in range(self.K):
        cat = torch.cat([h[src], h[dst]], dim=-1)
        e_ij = F.leaky_relu(
            self.W_att[k](cat)).squeeze(-1)
        alpha = torch.zeros_like(e_ij)
        for j in dst.unique():
            mask = (dst == j)
            alpha[mask] = F.softmax(e_ij[mask], dim=0)
        h_new = torch.zeros(
            h.size(0), self.W[k].out_features,
            device=h.device)
        Wh = self.W[k](h[src])
        h_new.scatter_add_(0,
            dst.unsqueeze(-1).expand_as(Wh),
            alpha.unsqueeze(-1) * Wh)
        outs.append(h_new)
    return torch.cat(outs, dim=-1)

```

A.3 EMA Memory Module

```

class EMAMemory:
    def __init__(self, num_nodes, embed_dim, alpha=0.5):
        self.alpha = alpha
        self.memory = torch.zeros(num_nodes, embed_dim)

```

```

def update(self, h_new):
    self.memory = (self.alpha * h_new
                   + (1 - self.alpha) * self.memory)
    return self.memory

```

A.4 Self-Supervised Training Loop

```

import torch.optim as optim
from sklearn.metrics import roc_auc_score

def train_hatcom(model, graph, pos_edges, neg_edges,
                 epochs=5, lr=0.001, batch_size=512):
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.BCEWithLogitsLoss()
    all_edges = torch.cat([pos_edges, neg_edges], dim=0)
    labels = torch.cat([torch.ones(len(pos_edges)),
                       torch.zeros(len(neg_edges))], dim=0)

    n = len(all_edges)
    idx = torch.randperm(n)
    train_idx = idx[:int(0.8 * n)]
    val_idx = idx[int(0.8 * n):]

    for epoch in range(epochs):
        model.train()
        h = model.encode(graph)
        total_loss = 0
        for i in range(0, len(train_idx), batch_size):
            batch = train_idx[i:i+batch_size]
            u, v = all_edges[batch, 0], all_edges[batch, 1]
            loss = criterion(
                (h[u] * h[v]).sum(dim=-1), labels[batch])
            optimizer.zero_grad()
            loss.backward()

```

```

        optimizer.step()
        total_loss += loss.item()
    model.eval()
    with torch.no_grad():
        h_v = model.encode(graph)
        u_v = all_edges[val_idx, 0]
        v_v = all_edges[val_idx, 1]
        scores = torch.sigmoid(
            (h_v[u_v] * h_v[v_v]).sum(dim=-1))
        auc = roc_auc_score(
            labels[val_idx].numpy(), scores.numpy())
    print(f"Epoch {epoch+1}: loss={total_loss:.4f},"
          f" val_auc={auc:.4f}")
return model

```